

Desafío I: Sweet Crush

EMILIANO DE JESUS LINCE DIAZ

Facultad de Ing. Electrónica UdeA

Correo: dejesus.lince@udea.edu.co

ESTEBAN GARCES ATEHORTUA

Facultad de Ing. Electrónica UdeA

Correo: esteban.garces2@udea.edu.co

I. ANALISIS DEL PROBLEMA Y CONSIDERACIONES

El desafío exige implementar un juego llamado (Sweet Crush) en lenguaje C++ en el framework Qt. El juego consiste en fichas que están representadas exactamente por 3 bits, empaquetados de forma continua en memoria dinámica, sin poder usar struct, class, template, STL ni string. Esto obliga a resolver el problema de dos maneras: una con acceso a bits (indexación y operaciones bitwise) y la segunda de lógica de juego (combinaciones, cascadas, redimensionamiento), ambas independientes de la interfaz gráfica.

Analizando el reto, identificamos los siguientes retos principales de desarrollo del juego:

- Leer/escribir 3 bits que pueden estar repartidos entre dos bytes consecutivos.
- Redimensionar la memoria (sin sobredimensionar), incluso cuando se inserte/elimine filas o columnas en posiciones intermedias.
- Procesar las cascadas simultaneas sin duplicar eliminaciones.

II. ESQUEMA DE TAREAS Y DISEÑO MODULAR

El desarrollo del programa se dividió de manera modular, usando 4 módulos independientes que garantizaran esto distribuidos de la siguiente manera:

II-A. Módulos para funciones

Para el código usamos 4 módulos como lo habíamos planteado en el preinforme, cada uno con una función diferente a los demás, pero para la solución implementamos algunos cambios en esta parte, pasamos las funciones de agregar/eliminar fila/columna al modulo de manejo de memoria dado que en estos casos, implementamos la expansión y reducción de memoria dentro de estas funciones, por lo cual también se eliminan las funciones de expandir memoria, reducir memoria de nuestro diseño final, también incluimos una nueva función en el módulo logica de juego llamada rellenar, la cual utilizamos para rellenar el tablero luego de cada combinación realizada, ya que nuestra funcion de agregar fichas aleatorias solo la usamos para generar el tablero ya que en ella generamos el tablero

sin combinaciones posibles realizando validaciones horizontales y verticales reemplazando las fichas que puedan generar una combinación. En el módulo de interfaz de usuario, agregamos también algunas funciones, las cuales sirven para mostrar el menú del juego que ve el usuario al momento de jugar. Con estos cambios realizados en el diseño del juego nuestros 4 módulos quedan de la siguiente manera:

- **Módulo 1 (Motor de bits):** En este módulo se realizarán todas las operaciones matemáticas y de bits, solo se usará para leer y escribir en la memoria las fichas. Este módulo tendrá las siguientes funciones:
 - **Función calcular_Byte_Desplazamiento:** Dada la fila y la columna se encargará de calcular en qué byte de la memoria está la ficha y qué desplazamiento tiene.
 - **Función leer ficha:** Extrae de la memoria los 3 bits de la ficha teniendo en cuenta los 3 casos posibles que se pueden dar.
 - **Función escribir ficha:** Modifica los bits de la ficha en la memoria sin modificar ningún otro bit vecino.
- **Módulo 2 (Manejo de memoria):** Este módulo se encarga del manejo de la memoria para que el juego funcione correctamente y respete la regla del 65 %. En este módulo está el cambio principal explicado previamente:
 - **Función inicializar tablero:** Calcula los bytes necesarios para el tablero y pide la reserva de la memoria.
 - **Funciones de agregar/eliminar fila/columna:** Se encargan de reorganizar el tablero luego de la elección del usuario haciendo los movimientos de bits necesarios. Además en estas funciones se incluyen la expansión y la reducción de la memoria en caso de ser necesario usando la regla del 65 %.
- **Módulo 3 (Lógica del juego):** Este módulo se encarga de las reglas del juego:
 - **Función generar fichas aleatorias:** Llenará las posiciones vacías del tablero con fichas al azar de manera uniforme.
 - **Función buscar combinaciones:** Recorre las

filas y columnas del tablero buscando las posibles combinaciones de 3 o más fichas.

- **Función eliminación por combinación:** Elimina las fichas validadas y aplicará la gravedad para que caigan nuevas fichas a esa posición.
- **Función cascadas:** Detecta si al caer las nuevas fichas se generaron nuevas combinaciones.
- **Función eliminación por usuario:** Recibe la coordenada dada por el usuario y elimina esa ficha, también activa la gravedad para nuevas fichas.
- **Función detectar cruces simultáneos:** Detecta si generan combinaciones tipo T, L, etc. Las elimina y activa gravedad para fichas nuevas.
- **Función gravedad:** Después de cada eliminación se activa para rellenar los espacios vacíos del tablero.
- **Función rellenar:** Se encarga de rellenar el tablero con fichas aleatorias luego de las eliminaciones, se genera para que el juego sea mas atractivo y genere mas cascadas como se explico anteriormente.
- **Módulo 4 (Interfaz Usuario):** Este módulo se encarga del manejo de los tableros y menús para mostrarlos en consola:
 - **Función tablero fichas:** Muestra el tablero de juego con las fichas, especial para el jugador.
 - **Función tablero binario:** Muestra el tablero de juego en binario, especial para el profesor.
 - **Función puntuación:** Muestra la puntuación junto con los movimientos y cascadas realizadas.
 - **Función mostrar menú partida:** Muestra el menú principal del juego con el que el usuario interactúa, en este se muestran los posibles movimientos que el usuario puede realizar.
 - **Función mostrar ayuda:** Muestra una lista con comandos válidos y su función dentro del juego.

II-B. Estado del juego

Además de los módulos de las funciones, se crea un arreglo el cual se representa mediante variables individuales de tipo primitivo (no struct, según la restricción del desafío): filas actuales, columnas actuales, bytes reservados, eliminaciones realizadas por el usuario, fichas eliminadas en total, combinaciones detectadas, cascadas de la eliminación actual y puntuación acumulada. Estas variables se implementan en el main y se pasan entre funciones por puntero/referencia según se requiera actualizarlas.

III. ALGORITMOS IMPLEMENTADOS

En esta sección mostraremos los algoritmos usados durante el programa, la utilización de los menús, la codificación de las fichas y manipulación de los bits se mantienen tal cual habíamos diseñado inicialmente para el preinforme, ahora agregaremos en este caso los algoritmos usados para modificaciones estructurales del juego y para la lógica del juego.

III-A. Para todo menú

Todo menú debe incluir verificación de los datos ingresados (osea hacer validación), una opción de ayuda para comandos validos, y una opción de salir del juego (que siempre va estar disponible).

III-A1. Menú de inicio del juego:

- **Determinar el tamaño del tablero:** Pedir que ingrese cantidad de filas y cantidad de columnas por separado.

III-A2. Menú en partida:

- **Eliminar ficha:** Pedir al usuario las coordenadas en posición de la ficha a eliminar en la fila y columna correspondiente de dicha ficha.
- **Agregar fila:** Pedir al usuario la fila a agregar.
- **Eliminar fila:** Pedir al usuario la fila a eliminar.
- **Agregar columna:** Pedir al usuario la columna a agregar.
- **Eliminar columna:** pedir al usuario la columna a eliminar.

Se imprimirá en pantalla el tablero de juego, con su correspondiente representación grafica de cada ficha de juego y en formato binario.

IV. DISEÑO DE LA CODIFICACIÓN DE BITS

IV-A. Representación binaria de las fichas

Tabla I
REPRESENTACIÓN DE LAS FICHAS

Ficha	Representación bits	Tipo	R. gráfica
1	001	Normal	β
2	010	Normal	Ω
3	011	Normal	Σ
4	100	Normal	Γ
5	101	Normal	θ
6	110	Normal	μ
7	111	Especial	Estado Eliminación X
8	000	Especial	Estado vacío .

IV-B. Definición de los estados especiales

La ficha 8 que en su representación de bits es (000) representa el estado especial (vacío). La ficha 7 que en su representación de bits es (111) representa el estado especial (Eliminación).

- **Ficha 8:** Definimos que esta ficha va representar el estado (vacío), como una representación de que no hay fichas en esa posición.

- **Ficha 7:** Definimos que esta ficha va representar el estado (Eliminación), como una confirmación de que la ficha en la posición que está recorriendo se debe eliminar.

4. DISEÑO DE LAS FÓRMULAS DE ACCESO A BITS

Dada una posición (fila, columna) en un tablero de Filas $\times$ Columnas:

Tabla II
FÓRMULAS DE ACCESO A NIVEL DE BITS

Paso	Fórmula	Propósito
Índice lineal de la ficha	$I = \text{fila} \times \text{Columnas} + \text{columna}$	Convierte coordenadas 2D en una posición lineal.
Bit inicial de la ficha	$B = I \times 3$	Ubica el primer bit de la ficha en la tira continua.
Byte que la contiene	$\text{Byte} = \lfloor B/8 \rfloor$	Ubica el byte de memoria donde inicia la ficha.
Desplazamiento en el byte	$\text{Offset} = B \pmod{8}$	Bits de desplazamiento dentro de ese byte.
Bytes totales a reservar	$\text{Bytes} = \lceil ((3 \times \text{Filas} \times \text{Columnas}) + 7)/8 \rceil$	Redondeo hacia arriba (evita pérdida por truncamiento).

Offset puede tomar cualquier valor entre 0 y 7 (no solo 0-5). Sin embargo, solo cuando Offset está entre 0 y 5 la ficha cabe completa dentro de un único byte. Cuando Offset es 6 o 7, los 3 bits de la ficha quedan repartidos entre el byte actual y el siguiente, y deben tratarse como un caso adicional.

- **Offset 0-5 (caso simple):** desplazar el byte a la derecha (offset) bits, aplicar máscara 00000111 (valor 7) con AND para leer; para escribir, invertir la máscara desplazada a la posición del offset con NOT, aplicar AND con el byte original para limpiar los 3 bits, y luego OR con la ficha nueva ya desplazada al offset.
- **Offset 6-7 (caso de cruce entre bytes):** leer los bits que quedan disponibles al final del byte actual y combinarlos, mediante desplazamientos y OR, con los bits que faltan al inicio del byte siguiente; para escribir, aplicar el mismo procedimiento de máscara/limpieza en ambos bytes por separado, sin alterar los bits vecinos de ninguno de los dos.

IV-B1. Redimensionamiento del tablero: Para agregar filas y columnas utilizamos la técnica de copiar y redimensionar, para esto recorremos el nuevo tablero y calculamos nuevamente el origen de los datos ajustando así el tablero para evitar que los bits queden desalineados al insertar los nuevos bits.

IV-B2. Detección de mascarar de bits: Para poder identificar las combinaciones simultaneas y cruces, creamos dos mascarar y utilizamos la operación AND lógica entre ellas, si el resultado es 1 se confirma el cruce y se procede a eliminarse.

V. PROBLEMAS DE DESARROLLO

Durante la fase de pruebas del código, tuvimos problemas de los que se van a detallar a continuación:

V-A. Inconsistencia y Doble Incremento en el Arreglo de Estado

V-A0a. Descripción del Problema:: Durante las primeras iteraciones de prueba, se detectó un fallo crítico en la precisión de las estadísticas del juego. Al iniciar una partida o ejecutar un movimiento, ciertos indicadores del arreglo estadoJuego (como el conteo de movimientos y fichas eliminadas) no incrementaban, o registraban sumas erróneas solo en el primer turno.

V-A0b. Análisis de Causa Raíz:: Al realizar la depuración de la traza de ejecución, se identificó que el arreglo de estado se encontraba duplicada y fragmentada en dos puntos distintos de la arquitectura:

1. **Capa de Control (main.cpp):** En el ciclo principal de ejecución y dentro de la estructura switch-case de las opciones del menú, se realizaban incrementos directos sobre los índices del arreglo estadoJuego antes o después de invocar los módulos del juego.
2. **Capa de Lógica (logicaJuego.cpp):** De forma paralela, las funciones internas (tales como eliminarFicha, funcion_eliminacion_por_combinacion y funcion_cascadas) también modificaban el arreglo mediante el puntero int* estadoJuego para contabilizar las eliminaciones, combos y puntos acumulados.

Este desacoplamiento de la distribución de responsabilidades provocaba efectos secundarios (side effects), generando redundancia en la actualización de los datos del juego al procesar un mismo evento desde dos capas distintas.

V-A0c. Solución Aplicada:: Para resolver el inconveniente y garantizar la integridad de los datos, se reestructuró la arquitectura del código aplicando el principio de separación de responsabilidades:

- **Centralización en la Lógica de Juego:** Se trasladó la responsabilidad total de los datos de estadoJuego hacia el módulo logicaJuego.cpp. Funciones como eliminarFicha() y funcion_cascadas() pasaron a ser las únicas encargadas de auditar y actualizar las estadísticas en memoria.
- **Purificación de main.cpp:** Se eliminó cualquier intento de modificación manual o directa del arreglo estadoJuego dentro del archivo main.cpp, relegando a este último la única función de orquestar el flujo y transferir la referencia del puntero estadoJuego a los procedimientos correspondientes.

V-A0d. Resultado:: La eliminación del doble punto de modificación solucionó el bug de manera definitiva, garantizando que el estado del juego se mantenga 100 % consistente y preciso durante todo el